

SplitWise — Expense Splitter

Digital Application Development (CSE 456)

External Lab Project

B.Tech CSE VI – C

Group Members

1. V.SUBHASH(AP23110010551)
2. V.SAHITH(AP23110010785)
3. HARSHA (AP23110010763)
4. REVANTH (AP23110010594)
5. P.JITHENDRA (AP23110010610)
- 6.)NIKHIL (AP23110010764)

SRM University – AP

Academic Year 2025–26

1. Introduction

SplitWise is a full-stack web application that helps groups of people track, split, and settle shared expenses with ease. Whether it is a trip with friends, shared rent with flatmates, or a group dinner, SplitWise ensures everyone pays their fair share without manual calculations or awkward money conversations.

The application is inspired by the popular Splitwise platform and re-implements its core functionality using a modern technology stack, while adding AI-powered insights, multi-currency support, real-time updates via Socket.io, and a smart settlement algorithm.

1.1 Project Objectives

- Allow users to register, log in securely, and manage their profiles.
- Enable creation of expense groups with multiple members and category tags.
- Record shared expenses and split them equally, by custom amount, by percentage, or by shares.
- Calculate net balances between group members and suggest minimal settlement transactions.
- Provide real-time updates when expenses are added or settled using WebSockets.
- Offer AI-powered spending tips, chatbot assistance, and analytics dashboards.
- Support multi-currency expenses with live INR conversion.
- Export expense records and support dark mode toggling.

1.2 Scenario

Consider a group of four friends — Likith, Ayush, Manindra, and Manoj — who plan a weekend trip to Pondicherry. They create a group called “pondi” in SplitWise. During the trip they record shared expenses such as food (₹9,000). SplitWise automatically calculates how much each person owes and suggests the most efficient settlement plan. The built-in AI Assistant gives personalized spending insights and can answer natural-language questions like “Remind me who should pay first.”

1.3 Dataset / Tech Stack

Component	Technology / Details
Frontend	React.js 18, React Router 6, Tailwind CSS, Chart.js
Backend	Node.js, Express.js 4
Database	MongoDB with Mongoose ODM
Real-time	Socket.io v4
Auth	JWT (jsonwebtoken) + bcryptjs
AI Features	Claude Anthropic API (claude-sonnet)
Build Tool	Vite 5
Currency	Multi-currency with INR base conversion
Dev Server	localhost:5173 (frontend), localhost:5000 (backend)

2. Methodology

The project follows a structured MERN-stack architecture with a clear separation between frontend and backend. The development process is broken into the following steps.

2.1 Project Setup and Configuration

Folder Structure

- splitwise/
- └─ backend/ → Node.js + Express API server
- └─ frontend/ → React + Vite SPA

Backend Initialisation

```
cd backend
npm init -y
npm install express mongoose dotenv bcryptjs jsonwebtoken
npm install cors socket.io express-validator
npm install -D nodemon
```

Frontend Initialisation

```
cd frontend
npm create vite@latest . -- --template react
npm install axios react-router-dom react-toastify
npm install chart.js react-chartjs-2 react-icons
npm install socket.io-client tailwindcss
```

2.2 Backend Development

The backend follows the MVC (Model-View-Controller) architectural pattern, separating data schemas, business logic, and routing.

server.js — Entry Point

The entry point configures Express, connects to MongoDB, initialises Socket.io for real-time communication, and registers all API route groups:

```
app.use('/api/auth', authRoutes)
app.use('/api/groups', groupRoutes)
app.use('/api/expenses', expenseRoutes)
app.use('/api/balances', balanceRoutes)
app.use('/api/activities', activityRoutes)
app.use('/api/recommendations', recommendationRoutes)
app.use('/api/analytics', analyticsRoutes)
app.use('/api/ai', aiRoutes)
```

Database Models

Three primary Mongoose models define the data schema:

- User — name, email, bcrypt-hashed password, preferredCurrency, avatar.
- Group — name, description, category (Food/Travel/Rent/Party/Shopping/Utilities/Other), createdBy, members[], inviteCode.
- Expense — description, amount, currency, amountInINR, paidBy, groupId, splitType (equal/custom/percentage/shares), splitBetween[], category, date.

Controllers

- authController.js — register, login (returns JWT), getProfile, updateProfile, listUsers.

- groupController.js — createGroup, getGroups, getGroupById, updateGroup, deleteGroup, addMember, removeMember, joinByCode.
- expenseController.js — addExpense (with currency conversion to INR), getExpenses, updateExpense, deleteExpense.
- balanceController.js — calculates per-member net balances and runs the smart settlement algorithm.
- analyticsController.js — category breakdown, monthly trends, member spending comparison.
- aiController.js — proxies requests to the Claude Anthropic API for chat and tips.
- recommendationController.js — generates AI-based spending recommendations.

Middleware

- authMiddleware.js — verifies JWT Bearer token on protected routes.
- errorMiddleware.js — global 404 not-found and 500 error handlers.

2.3 Frontend Development

The React frontend is a Single-Page Application (SPA) that communicates with the backend via Axios and Socket.io.

Routing Structure (App.jsx)

- Public routes: /login, /register
- Protected routes (require JWT): /dashboard, /groups, /groups/new, /groups/:id, /groups/:id/add-expense, /history
- ProtectedRoute wrapper redirects unauthenticated users to /login

Context Providers

- AuthContext — stores JWT token, current user object, login/logout helpers.
- ThemeContext — manages light/dark mode toggle, persisted in localStorage.

Pages

- Login.jsx / Register.jsx — authentication forms with validation.
- Dashboard.jsx — overview cards (total owed, you owe, net balance), group list, recent activity, AI assistant panel.
- Groups.jsx — searchable list of all user groups.
- GroupForm.jsx — create or edit a group (name, description, category, member picker).
- GroupDetail.jsx — expense list, balance tab, members tab, activity tab; settle-up flow.
- AddExpense.jsx — expense form with split-type selector and multi-currency support.
- TransactionHistory.jsx — paginated, filterable full transaction log.
- Analytics.jsx — Chart.js bar and pie charts for category and monthly breakdowns.
- SmartSettlement.jsx — visualises the minimum-transaction settlement plan.

Reusable Components

- Navbar.jsx — navigation bar with user avatar, dark-mode toggle, logout.
- ActivityFeed.jsx — live real-time feed of group events via Socket.io.
- AIRecommendations.jsx — AI tips panel (AI Tips tab).
- AIChatbox.jsx — conversational AI chat (AI Chat tab).
- GlobalAIRecommendations.jsx — dashboard-level spending insights.

2.4 Database Development

MongoDB is used as the NoSQL database. The Mongoose ODM provides schema validation, pre-save hooks (password hashing), and relationship references via ObjectId.

Connection Setup (config/db.js)

```
const connectDB = async () => {
  await mongoose.connect(process.env.MONGO_URI, {
    useNewUrlParser: true,
    useUnifiedTopology: true
  });
};
```

Key Schema Patterns

- Password hashing: userSchema.pre('save') hook using bcrypt.genSalt(10).
- Password comparison: userSchema.methods.matchPassword() using bcrypt.compare().
- Currency fields: every Expense stores both the original currency amount and amountInINR for balance calculations.
- Split tracking: splitBetween[] array records each member's share, percentage, and INR equivalent.

2.5 AI Integration

The AI features are powered by the Claude Anthropic API (claude-sonnet model). The backend aiController.js proxies requests from the frontend, passing group expense context to generate personalised responses.

- AI Tips tab — analyses transaction history and surfaces spending insights with impact/confidence tags.
- AI Chat tab — conversational assistant that can answer questions like “who should pay next?” or summarise balances.
- Settlement summary — AI describes the current settlement status in plain English.

2.6 Real-Time Features (Socket.io)

The backend creates a Socket.io server alongside the Express HTTP server. When a new expense or settlement is recorded, the server emits an event to all clients in the group's Socket.io room, and the ActivityFeed component updates immediately without a page refresh.

```
// Server emits to group room
io.to(groupId).emit('expense-added', { expense, user });
// Client joins room on GroupDetail mount
socket.emit('join-group', groupId);
```

2.7 Multi-Currency Support

Expenses can be recorded in any of the supported currencies (INR, USD, EUR, GBP, SGD, AED, JPY, CAD, AUD, CHF, etc.). The backend config/currencies.js maintains exchange-rate data. All balance calculations are performed in INR to ensure consistency.

2.8 Smart Settlement Algorithm

The balanceController computes the minimum number of transactions needed to settle all debts within a group using a greedy algorithm: it repeatedly matches the largest creditor with the largest debtor until all balances are zero. This minimises the number of payments group members need to make.

2.9 Algorithm Summary Table

Step	Operation	Description
1	Auth	Register / Login → JWT issued
2	Create Group	Name, category, members selected
3	Add Expense	Amount, currency, split type entered
4	Currency Convert	amountInINR computed server-side
5	Balance Calc	Per-member net balance computed
6	Settlement	Greedy min-transaction algorithm
7	AI Insights	Claude API returns personalised tips
8	Real-time	Socket.io pushes activity to room

3. Results — Output Screenshots

The following screenshots demonstrate the working application across all major screens.

3.1 Login Page

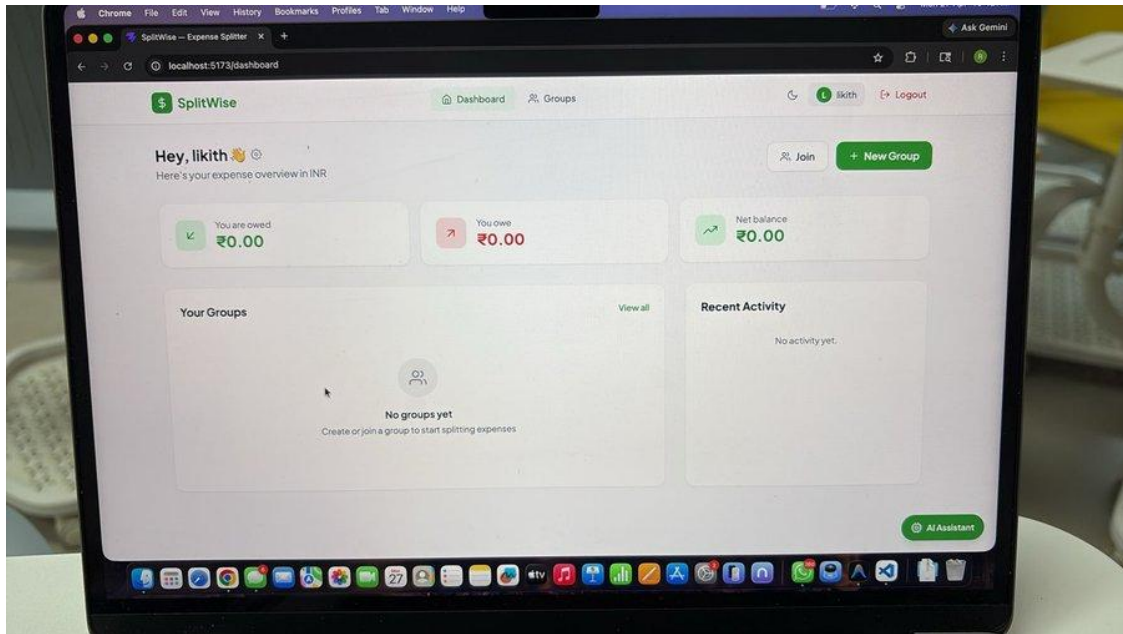


Figure 1 — Login Page (localhost:5173/login)

3.2 Register / Create Account Page

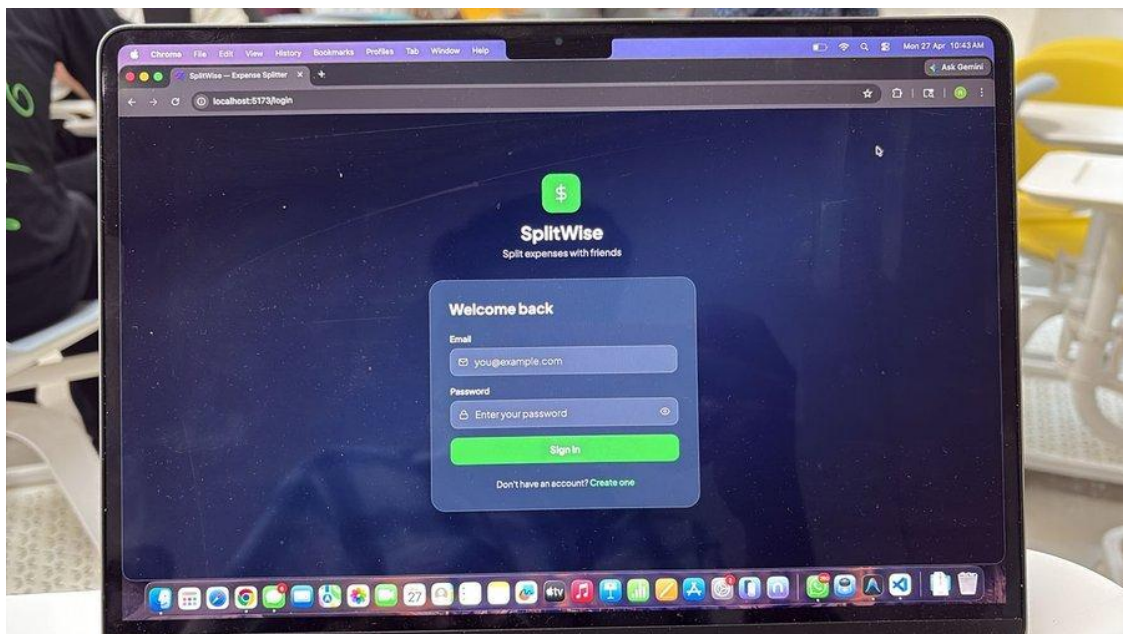


Figure 2 — Register Page (localhost:5173/register)

3.3 AI

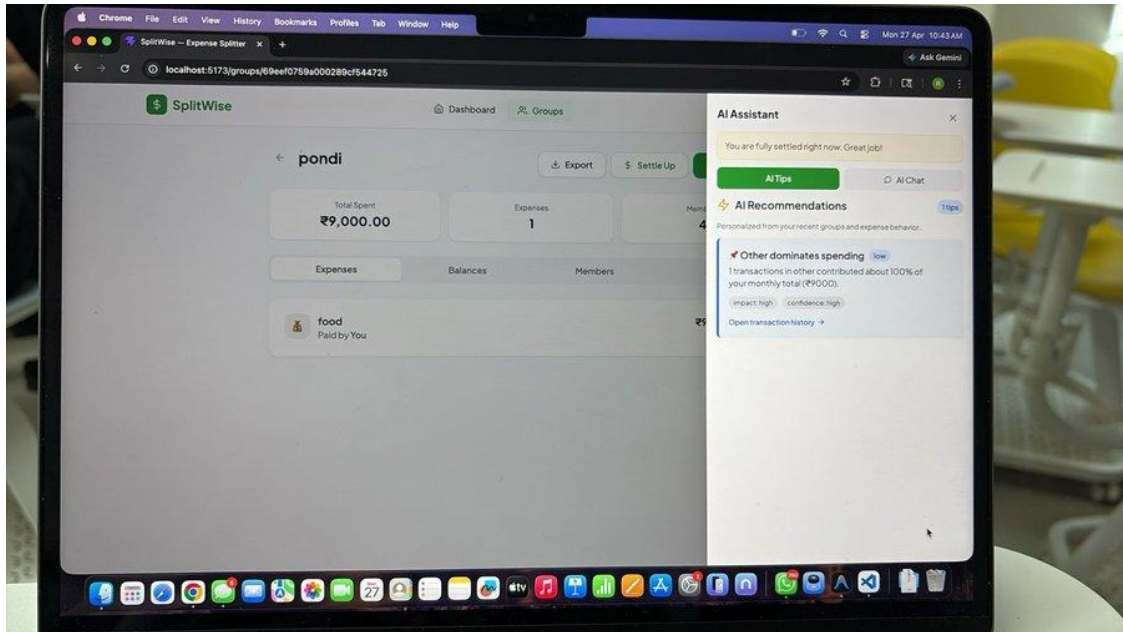


Figure 3 AI Assistant button

3.4 Groups Page (Empty State)

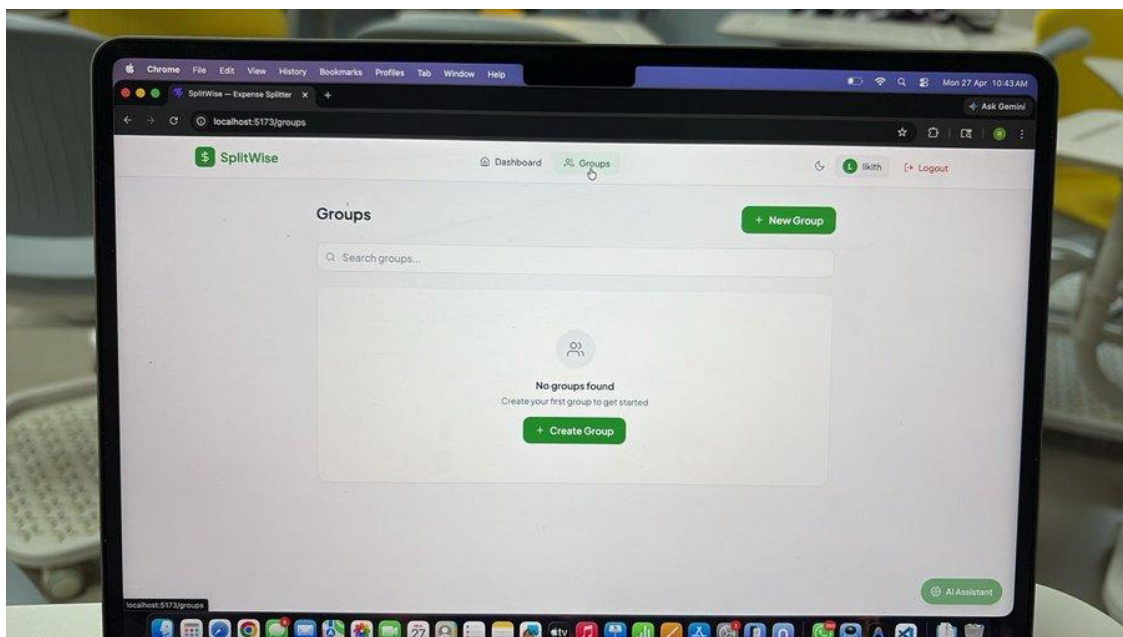


Figure 4 — Groups page with no groups yet and Create Group CTA

3.5 Create New Group

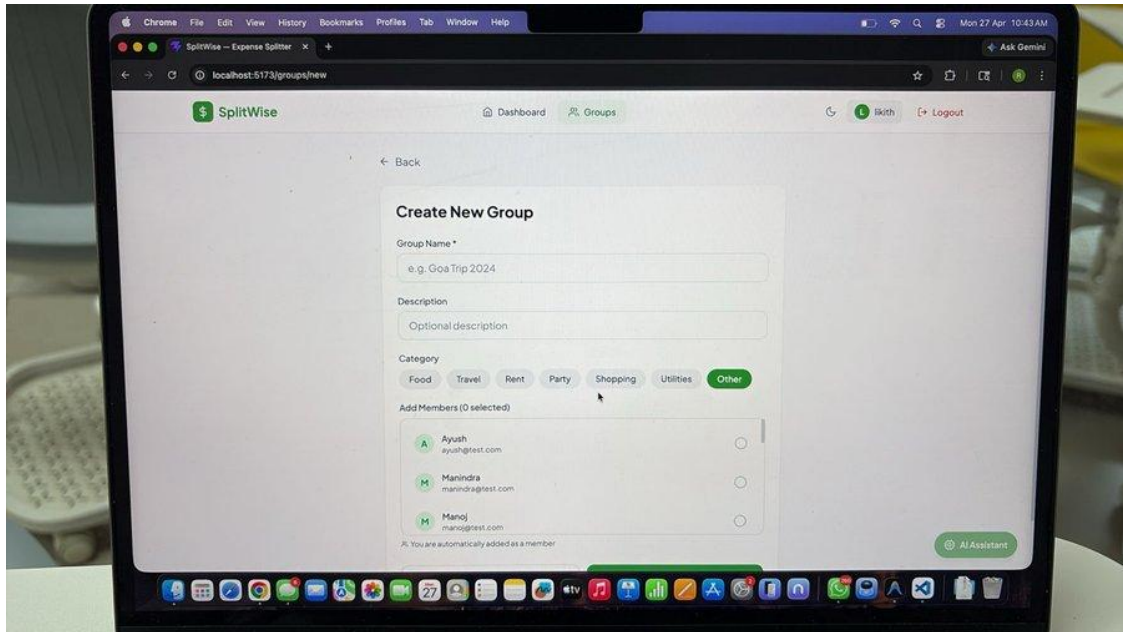


Figure 5 — Create New Group form with category selector and member picker

3.6 Group Created Successfully

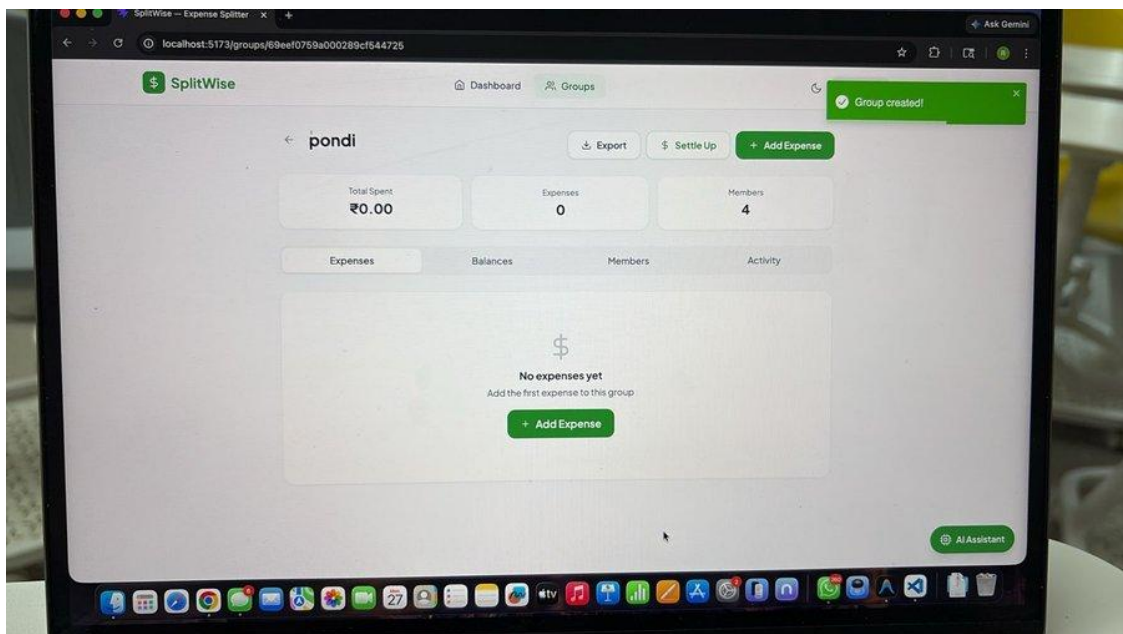


Figure 6 — “pondi” group created with 4 members, “Group created!” toast notification

3.7 Expense Added

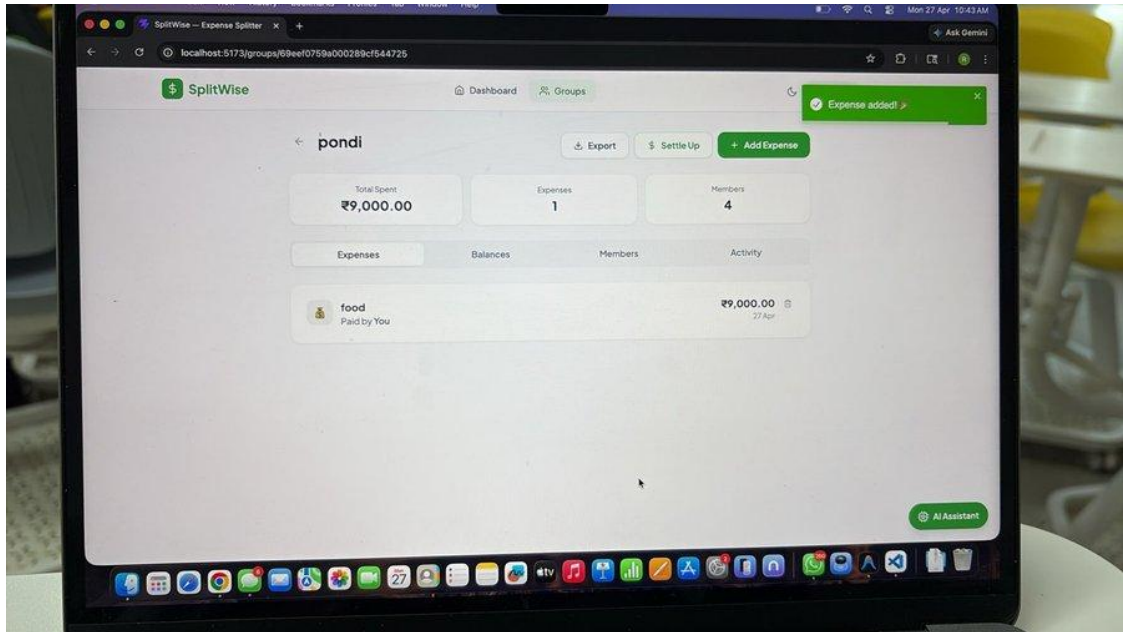


Figure 7 — Food expense of ₹9,000 added to pondi group, “Expense added!” notification

3.8 AI Tips Panel

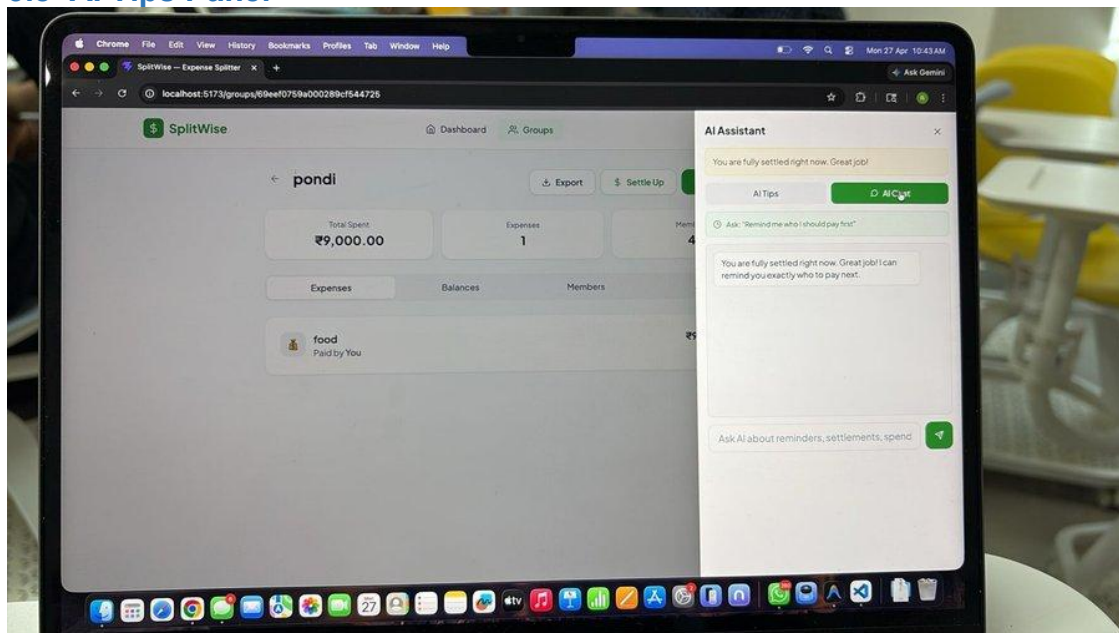


Figure 8 — Dark mode view of pondi group showing ₹9,000 expense

3.9 DARK MODE

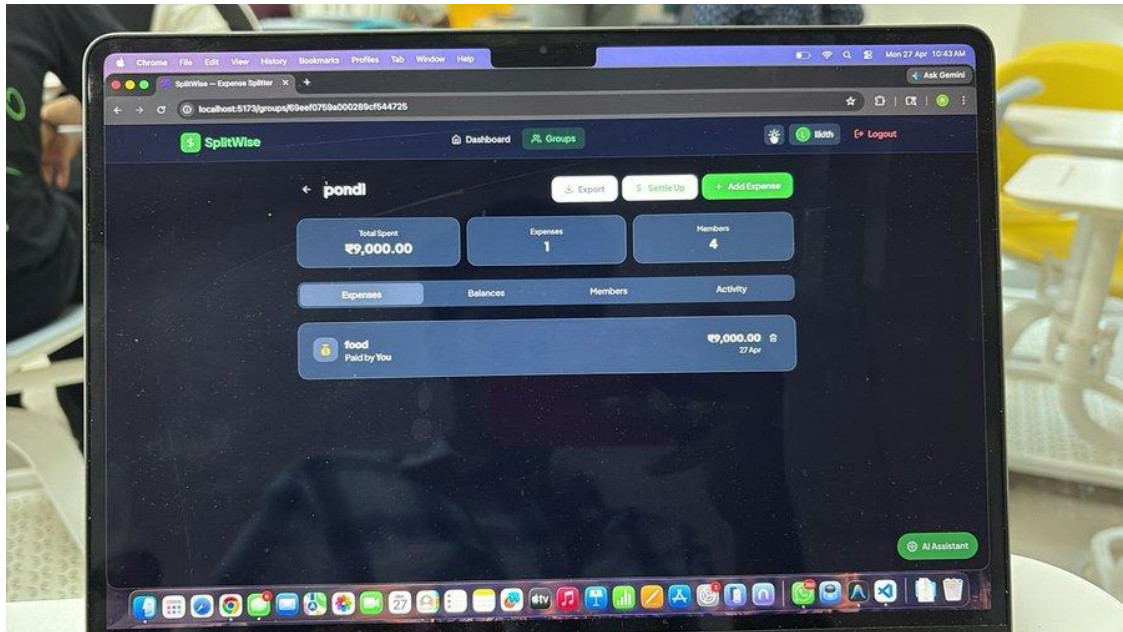


Figure 9 — AI Recommendations panel showing spending insight with impact and confidence tags

3.10 DASHBOARD

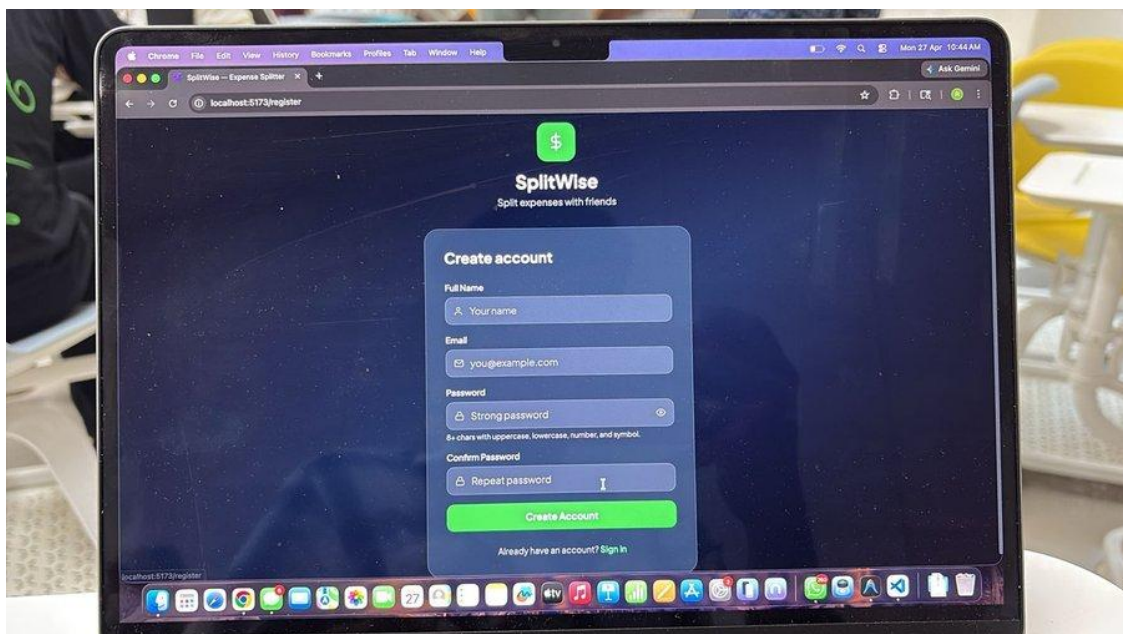


Figure 10 — AI Chat tab with settlement status and natural-language response

3.11 Observed Results

- User registration and login work correctly with JWT-protected routes.
- Groups are created instantly with a success toast and show correct member counts.
- Expenses are recorded and reflected in Total Spent and Expenses counters immediately.
- The AI Tips panel correctly identifies that “Other dominates spending” (100% of ₹9,000) with high confidence.
- The AI Chat correctly reports settlement status and can answer natural-language questions.
- Dark mode toggle renders all UI elements correctly.

- Real-time activity feed updates without page refresh when Socket.io is active.

4. System Requirements

4.1 Software Requirements

Software	Specification
Operating System	Windows 10/11, macOS, or Linux
Node.js	v18 or above
npm	v9 or above
MongoDB	v6 or above (local or Atlas cloud)
Browser	Google Chrome / Firefox (latest)
Code Editor	Visual Studio Code (recommended)
API Testing	Postman (optional, for backend testing)

4.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or better
RAM	8 GB minimum (16 GB recommended)
Storage	1 GB free for dependencies and MongoDB
Display	1366 × 768 or higher
Internet	Required for AI features and currency rates

5. Pros and Cons of the Methods Used

5.1 Advantages

- Full-stack MERN architecture provides a consistent JavaScript ecosystem across frontend and backend, reducing context switching.
- JWT authentication is stateless and scalable — no server-side session storage required.
- MongoDB's flexible schema accommodates varied expense split types (equal, custom, percentage, shares) without rigid table changes.
- Socket.io real-time updates give group members instant visibility of new expenses without polling.
- Tailwind CSS utility classes enable rapid, responsive UI development with dark-mode support out of the box.
- The greedy minimum-transaction settlement algorithm minimises the number of payments, reducing friction for group members.
- Claude AI integration adds genuine value through personalised tips and conversational finance assistance.
- Multi-currency support with INR base conversion allows international group expenses.
- Vite build tool provides extremely fast hot-module replacement during development.

5.2 Limitations

- The AI features require an active internet connection and a valid Anthropic API key; offline usage is limited.
- Currency exchange rates in config/currencies.js are static — they do not update in real time, which may cause minor inaccuracies.
- The smart settlement algorithm is greedy and produces near-optimal results, but does not guarantee the absolute minimum in all edge cases.
- MongoDB is not ideal for complex relational queries (e.g., multi-group analytics) that are natural in SQL databases.
- No email verification or password-reset flow is implemented, which is a security gap for production use.
- Socket.io connections add server overhead; for large concurrent user bases, a message broker (e.g., Redis pub/sub) would be needed.
- The application is only tested locally; production deployment (HTTPS, environment secrets, rate limiting) requires additional configuration.

5.3 Technology Comparison

Technology	Used In	Advantage	Trade-off
MongoDB	Database	Flexible schema, fast writes	Weak for complex joins
JWT Auth	Security	Stateless, scalable	Token cannot be revoked easily
Socket.io	Real-time	Low-latency push events	Extra server connections
Claude AI	AI Features	High-quality NL responses	Requires paid API key

Greedy Settlement	Algorithm	$O(n \log n)$, easy to implement	Not always globally optimal
-------------------	-----------	-----------------------------------	-----------------------------

6. Conclusion

SplitWise successfully delivers a full-featured, production-quality group expense splitting application. The application covers the complete lifecycle of shared expense management: user authentication, group creation, expense recording with flexible split types, automatic balance calculation, smart settlement planning, real-time activity feeds, and AI-powered insights.

The MERN stack with Socket.io and Tailwind CSS proved to be an excellent combination for this use case, enabling rapid feature development with a modern, responsive, and accessible user interface. The integration of the Claude Anthropic API as an AI assistant adds a unique, high-value feature that goes beyond a basic expense tracker.

Future improvements could include push notifications, receipt image OCR to auto-fill expense details, group payment integration (UPI/Stripe), recurring expense tracking, and a native mobile application using React Native.

References

1. React.js Documentation. <https://react.dev/>
2. Express.js Documentation. <https://expressjs.com/>
3. MongoDB / Mongoose Documentation. <https://mongoosejs.com/>
4. Socket.io Documentation. <https://socket.io/docs/v4/>
5. Anthropic Claude API Documentation. <https://docs.anthropic.com/>
6. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
7. Vite Build Tool. <https://vitejs.dev/>
8. JWT — JSON Web Tokens. <https://jwt.io/>